

1. このシステムを開発する経緯・目的

- 前回のプロダクトでは以下の課題が残った。
 - ・いつてしまえば従来のシステムのリファクタリングで、労力削減という目的を達成できなかった。
 - ・AIを使って楽をしたいという目的に添えなかった。
 - ・機械系は、学習コストがかかるため学習コストの少ないものを作成したい。

課題の解決ができると

- ・労力を削減するためにスケジューリング問題の解法やAIを活用し、30分以内でシフト作成を行えるようにする。
- ・通常業務に集中させることで利益増加を見込める。
- ・学習コストが少なく操作できる。またミスによるシステムの破壊を防げる。担当者が変更しても従来システムを使用することができる。

2. プロジェクト名

AI-Shifts-Management(仮決定の名前)

3. 開発スコープ

F-1: AIシフト機能

- ・生成AIとCP-Satによる制約プログラミングによってシフトを作成する。
- ・生成AIはその月特有の事情を自然言語で入力することで制約を判断し、CP-satでは従業員ごとの制約をすべて組み合わせた結果から答えを出して、それをシフトにする。生成AIは自然言語を処理し、それをJsonにして制約などにする。CP-satでは実際にスケジューリング問題を解いてもらう。

F-2: 人件費自動計算機能

- ・全体の1ヶ月の人件費とその日1日の人件費を計算する。
- ・労働時間から社会保険・雇用保険を適用するかしないかを判断。

4.画面・機能要件

- ログイン画面
シフト管理者、一般社員でできる機能を制限したり、赤の他人がサイトに間違ってきた際に中を見られないようにしたりするため。Google Workspace認証を想定している。(OAuth認証)
 - ・ログインの後には、ダッシュボードへと遷移する。
- ダッシュボード
ログイン後に最初に表示する画面。今月のシフトが表示されている。各機能の入り口ともなっている。
 - ・1ヶ月の人件費を表示する。
 - ・その従業員個人の総勤務時間を表示
 - ・シフト管理者・一般社員がこの画面ではシフトは編集できない(表示するだけ)
 - ・26日になったら通知する
 - ・シフトを編集するボタンを表示して押したら、シフト表画面に遷移する。

- 従業員管理画面

従業員の情報について詳細に入力できる。

- 入力できる項目

・Google Workplaceのメールアドレス。ここで登録されれば、従業員も使用することができるようになる。登録されなくても表示はされるようにする。

- ・名前
- ・年齢
- ・交通費
- ・メインで入るシフト
- ・週に何回入るか

・ルール設定画面

- シフト表画面(メイン)

まず最初の画面として、シフト管理者が各従業員の休日希望(絶対的勤務不可能日)・入りたいシフトを入力する。

その後シフト作成ボタンを押す。

ChatGPTなどが自然言語を整理したり、CP-satが色々な画面で入力されたハード制約とソフト制約を満たす決定変数を出力してシフトを出す。生成の進捗を表示する。

現時点でわかるハード制約:

必要人数の決め方

・曜日ごとに決める。

・金土日+祝日:朝3人 夜3人 深夜1人

・平日:朝2人 夜2人 深夜1人

シフト表が作成できた後、生成結果の表示と手動編集ができる中心画面に遷移する。曜日自動入力・土日祝の色分けをする(つまりカレンダーみたいにすればいい)。手動編集では該当するシフトの場所を押したら、プルダウンリストで選べるようにする。PDFとExcelファイルに出力できるようにする。PDFボタンを押すと印刷プレビュー画面に遷移する。Excelファイルも同様。

情報:

シフトパターン

基本・例外に別れる。

・基本 9時から17時(朝) 17時から1時(夜) 1時から9時(深夜)

・例外(二つ以上仕事を掛け持ちする人) 9~15. 9~16. 10~17. 10~18. 12~20. 15~23. 18~20、18~1、20~1

- 人件費計算画面

シフト表(ここからの変更不可)と人件費の計算結果を表示する。1日の人件費の結果と

一人一人の人件費の結果を出す。合計も出す(全て表形式)

保険の基準値

社会保障: 月120時間以上

雇用保険加入: 月80時間以上119時間まで

雇用保険未加入: 月79時間以内

割増(深夜料金など)含めるか 割増賃金は人件費に含める。

人件費は「その人に関わる全ての金額」

本来は有給、交通費が含まれる

Write me: 本人の作業ではなく、ここの計算ロジックを詰めて先方に聞く。想定従

業員数 大体25人

- 印刷プレビュー画面

PDFボタンとExcelボタンを押したら細かく編集できるようにする。

5. 共通要件

・ヘッダーでも画面の名前をかいて、現在いる画面をバーで表示する。

6. 使うプログラム言語やフレームワーク・ライブラリなど

・チーム開発を想定している。チーム開発に便利なツールは積極的に採用しよう。

- 技術スタック

層	技術	選定理由
フロント	Vue3	リアクティブな表に適合
状態管理	Pinia	Vueの公式ライブラリ
ルーティング	Vue Router	SPAで画面遷移
UI	Naive UI	表・カレンダーを効率構築
言語(フロントエンド)	TypeScript	型安全
API	FastAPI	OpenAPI自動生成
検証	Pydantic v2	入出力スキーマ検証
最適化	OR-tools CP-sat	シフトスケジューリング
ORM	SQLAlchemy 2.0 + Alembic	マイグレーション管理

DB	PostgreSQL	開発に便利
非同期処理	FastAPI BackgroundTasks	最適化計算をAPIから分離
認証	Google Workspace(OAuth) Magic Link or Supabase	パスワード管理の負荷を低減
祝日	JPholiday(py)	土日祝日判定
デプロイ	GCP + Terraform	楽だから

- チーム開発ツール群

・パッケージ管理:

フロントエンド: npm

バックエンド: uv

・環境構築ツール

Docker + docker-compose

devcontainer: VS codeの環境も統一

・Lint / Format

フロントエンド: ESLint + Prettier

バックエンド: Ruff + mypy

・Pre-commitフック

Pre-commit + husky + lint-staged

・テスト

FE: Vitest + Vue Test Utils

BE: Pytest + Pytest-cov

CI/CD

GCPで管理する。PR必須。

・シークレット管理:

.env + .env.exampleを構築

・ドキュメント管理

READMEの整理と、普段からWordドキュメントでの管理をする。